

Autonomous Pick-and-Place with a 4-DoF Arm

Abdullah Amir and Hamad Asif Khan

Habib University, Karachi, Pakistan

Email: aa08453@st.habib.edu.pk, hk08394@st.habib.edu.pk

Abstract—We describe a pick-and-place pipeline for the PhantomX Pincher 4-DOF arm that combines RGB-D perception, closed-form inverse kinematics, and a real-time MATLAB digital twin. An overhead Intel RealSense camera feeds a K-means color segmentation and watershed separation pipeline that localizes colored blocks in the robot base frame. A Denavit–Hartenberg-based analytical IK solver produces up to four candidate joint configurations per target; these are filtered for joint limits ($\pm 150^\circ$), self-collision along 20 interpolated waypoints, and floor contact, then ranked by a weighted displacement cost that penalises large base and shoulder motions. The system operates in two modes: a fixed straight-down approach ($\varphi = \pi/2$) for predictable gripping, and an auto-computed approach angle for broader workspace coverage. Over seven repeated pick-and-place trials the system achieved a mean placement error of 6.07 mm with a sample standard deviation of 5.18 mm, satisfying the ± 10 mm success criterion on six of seven trials. Perception correctly identified all four target colors across five validation scenes. The main challenges encountered were vibration on aging servos, workspace contraction under the fixed-orientation constraint, and sensitivity of the gripper calibration to cube-size variation. A parallel digital twin proved essential for resolving a firmware-offset discrepancy between the simulation and hardware.

Index Terms—PhantomX Pincher, robot manipulator, pick and place, Denavit–Hartenberg, inverse kinematics, RGB-D perception, point cloud

I. INTRODUCTION

Serial robotic manipulators are a mainstay of both industrial automation and educational robotics, performing tasks from assembly and sorting to palletizing with precision beyond human capability. For students of robotics, the ability to take a manipulator from a mathematical model through perception and planning to physical execution represents a complete traversal of the manipulation stack. The PhantomX Pincher, a compact four-degree-of-freedom arm built around Dynamixel AX-12A servo motors, offers a low-cost and reconfigurable platform on which this full pipeline can be explored [6].

This project addresses the following problem: given a target Cartesian position and an approach angle, move the end-effector of a 4-DOF arm to that pose, grasp an object, and deposit it at a second specified location—accurately, repeatably, and without collisions. When operating in autonomous mode, the pick location is not provided by a human but discovered by the perception subsystem, which detects and localizes colored blocks using overhead camera data. The integration of these capabilities—spatial perception, kinematic modelling, collision-aware motion planning, and gripper control—into a single reliable pipeline is the central challenge.

The kinematic model is formulated using the Denavit–Hartenberg (DH) convention [2], chosen because it is the

standard parameterization for serial chains, maps directly to MATLAB’s `rigidBodyTree` collision-checking utilities, and provides a systematic procedure for assigning coordinate frames to each joint. The inverse kinematics are solved in closed form by exploiting the arm’s planar structure: joints 2 through 4 share a common twist angle of zero, which reduces the problem to a base rotation followed by a two-link planar geometry solvable via the law of cosines. This yields up to four analytical candidate solutions per target, avoiding the convergence issues and computational cost of iterative numerical solvers. A real-time MATLAB digital twin was developed in parallel, mirroring every joint command on a 3D visualization, which proved indispensable for diagnosing coordinate-frame mismatches between the simulation and the Arbotix firmware.

This project was originally undertaken by a three-member team. Shaheer bin Tariq departed during the project period, and all final integration, testing, and reporting were completed by the remaining two members.

The remainder of this report is organized as follows. Section II describes the system architecture. Section III details the perception pipeline. Section IV covers kinematics and motion planning. Section V presents integration results. Section VI discusses failure modes and design evolution. Section VII concludes and outlines future directions.

II. SYSTEM ARCHITECTURE

A. Subsystem Overview

The robotic system is a pick-and-place pipeline consisting of a 4-DOF arm guided by overhead vision from an Intel RealSense camera. It is divided into three subsystems. The **Perception** subsystem acquires RGB and depth data from the camera and returns object poses in the robot’s base frame. The **Planning** subsystem receives these poses and computes collision-free joint-angle trajectories via the IK solver. The **Control** subsystem translates joint commands into physical servo actuation through the Arbotix-M controller.

B. Coordinate Frames and Conventions

Three coordinate frames are used throughout. The *base frame* $\{s\}$ has its origin at the projection of the first motor shaft onto the baseboard, with the z -axis pointing upward and the x - and y -axes following the right-hand rule. The *end-effector frame* $\{e\}$ is located at the centre of the gripper motor horn and maintains a consistent orientation with the base frame, parameterized by the approach angle φ . The *camera frame* $\{c\}$ has its origin at the RealSense optical centre, with z pointing

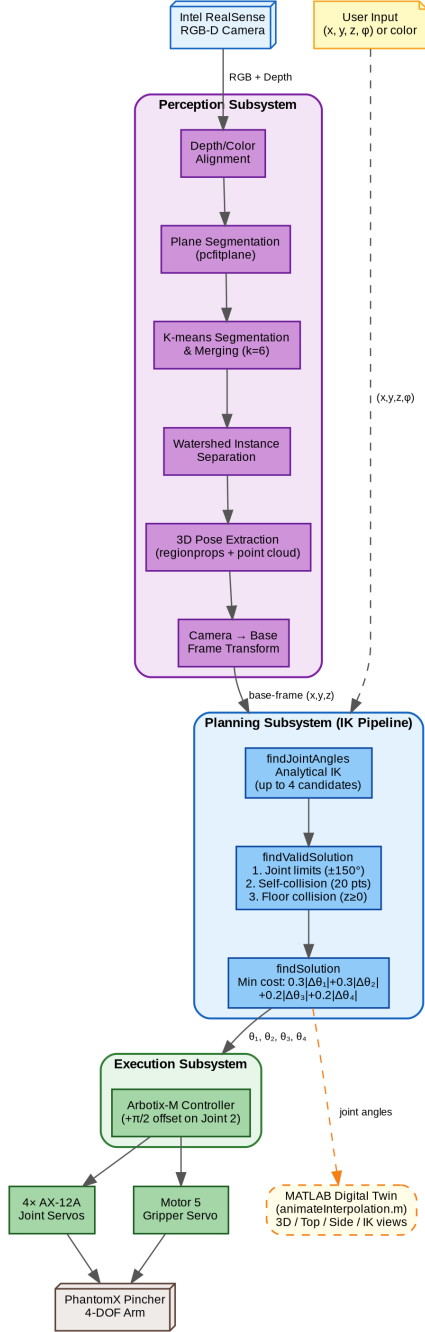


Fig. 1. Functional architecture of the robotic system. The three subsystems — Perception (purple), Planning (blue), and Execution (green) — communicate sequentially. The MATLAB digital twin (dashed) mirrors joint commands in parallel.

downward toward the workspace, x to the right of the image plane, and y running downward in the image plane.

The camera is mounted in a top-down configuration with a measured vertical standoff of approximately 700 mm above the baseboard. Points detected in the camera frame are transformed into the base frame via a calibrated rigid-body transform prior to being passed to the IK solver.

C. Kinematic Chain

The PhantomX Pincher has four revolute joints in an RRRR configuration: base pan about Z , shoulder pitch about Y , elbow pitch about Y , and wrist pitch about Y . Including the ground link, the arm comprises five links and four joints. The degrees of freedom are verified using Gruebler's formula with $N = 5$, $J = 4$, $m = 6$, and $f_i = 1$ for all joints:

$$\text{DoF} = m(N - 1 - J) + \sum_{i=1}^J f_i = 6(5 - 1 - 4) + 4 = 4 \quad (1)$$

Out of the six degrees of freedom required for full spatial pose control, only four are available, constraining the end-effector to a circular workspace and preventing independent control of roll and yaw.

The DH parameters are listed in Table I. Joint limits of $\pm 150^\circ$ are enforced in software, matching the rated range of the AX-12A servomotors. The nominal workspace is approximately cylindrical with radius ≈ 32 cm and height 0–45.7 cm above the base, though the effective reach under a fixed straight-down approach ($\varphi = \pi/2$) is reduced to approximately 21 cm.

TABLE I
PHANTOMX PINCHER DH PARAMETERS

Joint	a_i (cm)	α_i	d_i (cm)
1	0	$\pi/2$	13.7
2	10.5	0	0
3	10.5	0	0
4	11	0	0

D. Sensors and Actuators

The primary sensor is an Intel RealSense depth camera providing aligned 640×480 RGB and depth streams [7]. Pixel alignment ensures that coordinates (u, v) in the color image map to the same physical point in the depth map.

The actuators are five Dynamixel AX-12A servomotors, each offering a 300° range, 1 Nm stall torque, and 10-bit position resolution ($\approx 0.29^\circ/\text{step}$) [3]. Four servos drive the arm joints; the fifth drives the gripper. Inside each servo, a potentiometer is mechanically coupled to the output shaft: the slider tracks the shaft's rotation, producing a voltage proportional to the angular position, which the servo's internal controller reads for closed-loop position regulation.

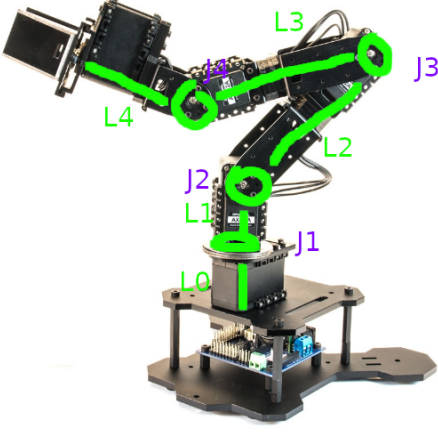


Figure 1.2: Phantom X Pincher Arm

Fig. 2. PhantomX Pincher with labeled joints (J1–J4) and links (L0–L4).

E. Static Payload Capacity

The torque at the first joint motor when the arm is fully extended is:

$$\tau = M_L g(L_1 + L_2 + L_3) + m_3 g(L_1 + L_2 + \frac{L_3}{2}) + m_2 g(L_1 + \frac{L_2}{2}) + m_1 g \frac{L_1}{2} \quad (2)$$

With $L_1 = L_2 = L_3 = 105$ mm, $m_1 = 106.1$ g, $m_2 = m_3 = 90.1$ g, and stall torque $\tau_s = 1.5$ Nm, the maximum payload (requiring no joint to exceed one-fifth of stall torque) is $M_L < 246.6$ g, consistent with the manufacturer’s specified capacity of ≈ 250 g [6].

F. Data Flow

The complete data flow is captured in Figure 1: the RealSense camera produces aligned RGB and depth frames, which enter the perception pipeline for color segmentation and 3D localization. The resulting base-frame coordinates are passed to `findJointAngles`, which computes IK candidates. These are filtered by `findValidSolution` and ranked by `findSolution`. The winning joint-angle vector is dispatched to the Arbotix controller (with a $+\pi/2$ firmware offset on joint 2), which actuates the servos. The gripper is controlled separately via a fixed motor-5 command.

G. Digital Twin

A MATLAB-based digital twin runs in parallel with the physical robot. The function `animateInterpolation.m` renders 20 linearly interpolated frames between any two joint configurations, calling `updatePlot` at each step. During early development, a systematic divergence was discovered between the simulation and the hardware: the firmware applies a $+\pi/2$ offset to joint 2 that the simulation did not initially replicate. Once this offset was mirrored in the simulation code, the digital twin reliably tracked the physical arm and became essential for validating motion plans.

III. PERCEPTION PIPELINE

A. Task Definition

The perception subsystem provides the robot with the spatial awareness necessary for autonomous manipulation. Given an overhead RGB-D view of the workspace, the pipeline identifies and localizes colored cuboid blocks, estimating each block’s 3D centroid and planar orientation in the robot’s base frame. The system handles four target colors (red, green, blue, yellow) and assumes that all blocks rest flat on the workspace, so only rotations about the z -axis need to be estimated.

B. Pipeline Architecture

The perception pipeline proceeds through six stages.

Stage 1 — Data Acquisition and Alignment. The Intel RealSense pipeline is initialized to stream aligned depth and color frames. The depth and RGB sensors are physically separated on the camera module; the alignment step ensures that a pixel at coordinates (u, v) in the color image corresponds to the same physical point at (u, v) in the depth map. This correspondence is critical for subsequent back-projection of 2D detections into 3D space.

Stage 2 — Plane Segmentation and Slicing. The function `find_plane.m`, a wrapper around MATLAB’s `pcfitplane`, identifies the dominant horizontal plane in the point cloud using a reference normal vector of $[0, 0, 1]$. This isolates the cube surfaces from the table, preventing the workspace from being misclassified as a colored object. A point cloud is generated via `pcfromdepth`, projecting 2D pixel coordinates and depth values into metric 3D coordinates (X, Y, Z) while mapping RGB color data onto each point.

Stage 3 — Color Segmentation and Merging. The function `segment.m` runs K-means clustering with $k = 6$ in RGB color space, deliberately over-segmenting the image so that no cube is missed. The function `mergeSegments.m` then computes the Euclidean distance between each cluster centroid and a set of reference RGB values for the four target colors (e.g., $\mathbf{r} = [87, 13, 17]$ for red). Clusters whose centroids fall within a threshold distance of the same reference are merged into a single unified mask. This two-step approach handles the common situation where shadows or specular highlights cause a single physical cube to split across multiple K-means clusters. The output is up to four binary masks, one per detected color category.

Stage 4 — Instance Segmentation. When multiple blocks of the same color are placed touching, they appear as a single connected component in the color mask. To separate them, the pipeline converts the image to Lab color space (which enhances perceptual color gradients) and applies marker-controlled watershed segmentation. This finds the boundary between touching objects of the same color, ensuring that each physical block receives a unique label. Without this step, the reported centroid would fall between the two blocks, causing the arm to grasp air.

Stage 5 — 3D Pose and Coordinate Extraction. For each labeled instance, MATLAB’s `regionprops` extracts

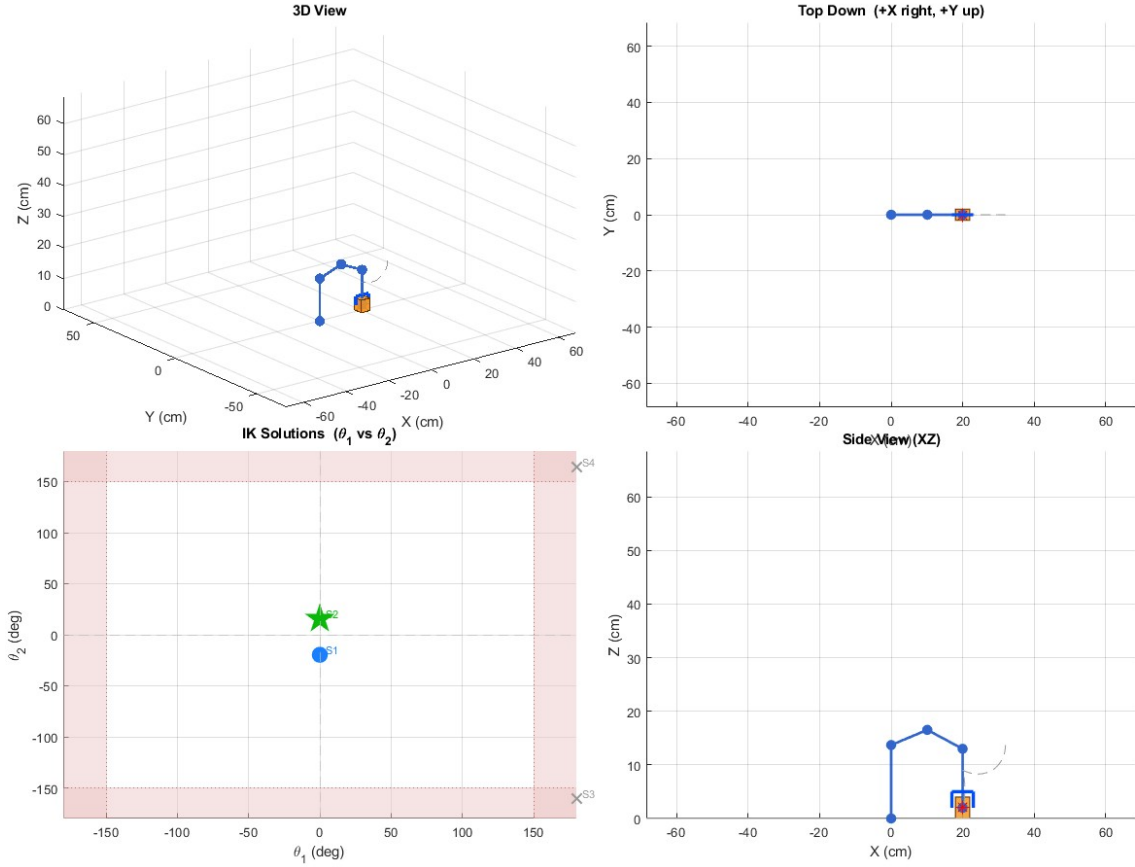


Fig. 3. MATLAB digital twin visualization. Top-left: 3D view of the arm reaching toward a target cube. Top-right: top-down projection (+X right, +Y up). Bottom-left: IK solution map (θ_1 vs θ_2) showing valid candidates (S1, S2) and rejected solutions (S3, S4; shaded region indicates joint limits). Bottom-right: side view (XZ) showing the vertical pre-grasp descent.

the 2D centroid (u, v) and the orientation angle of the blob's major axis (computed from the second moments of the binary region). The 3D centroid is obtained by querying the point cloud at the centre of each instance's bounding box, yielding metric coordinates $[X, Y, Z]$ in the camera frame. A 4×4 homogeneous transformation matrix is constructed from the 3D position and the z -axis rotation derived from the 2D orientation angle.

Stage 6 — Camera-to-Base Transform. Each centroid in the camera frame is transformed into the robot's base frame using the calibrated rigid-body transform. A fixed vertical offset is added to the z -coordinate to account for gripper contact height above the block's top surface. The transformed coordinates are then passed to the motion pipeline as the pick target.

C. Perception Results

The pipeline was validated across three datasets of increasing complexity: 25 RGB scenes (including stacked cubes, adjacent cubes, single cubes, and mixed-color arrangements under both overhead-light-on and off conditions), 5 point clouds used for primary algorithm development, and 27 point clouds for broader validation.

Testing was automated: each point cloud was loaded in sequence, the pipeline identified the color(s) and computed transformation matrices, and the output was compared against manually labeled ground truth. The over-segment-then-merge algorithm achieved a **5/5 pass rate** on the primary validation set, correctly identifying all target colors and computing valid transformation matrices for each detected block.

D. Perception Limitations

Several limitations were identified. The pipeline assumes only z -axis rotations, which holds for blocks resting flat but fails when cubes are stacked and subsequently topple—overlapping blocks with multiple visible faces produce incorrect labels and orientations. Same-color blocks that are fully stacked still merge into a single blob despite the watershed step. The K-means reference colors were tuned under standard lab lighting; performance degrades under variable illumination. Finally, the camera is invoked once at the start of each cycle—blocks moved by the arm are not re-detected until the next cycle begins.

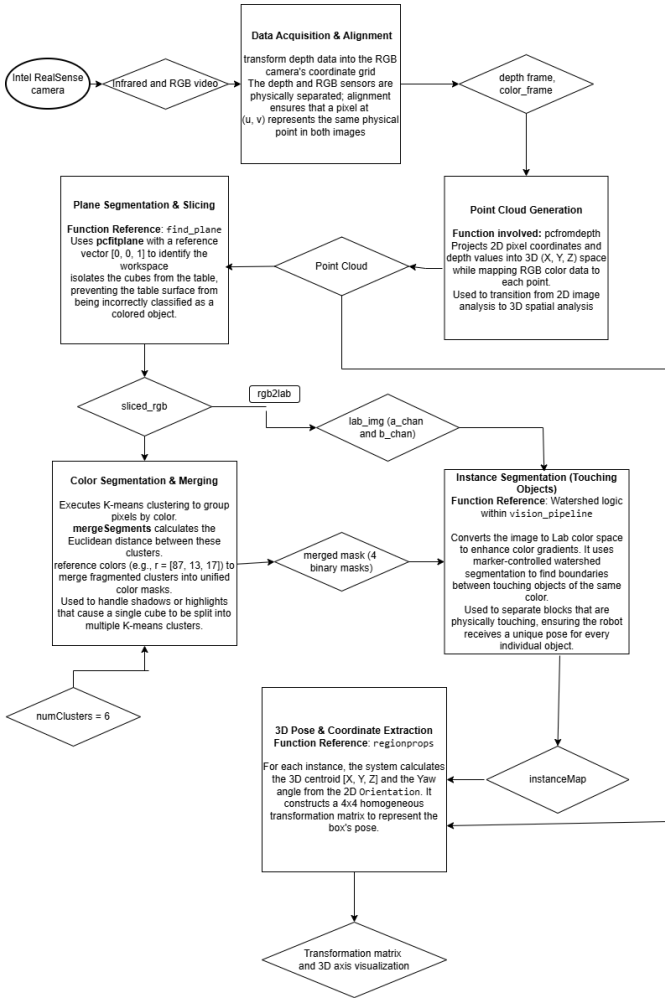


Fig. 4. Perception pipeline flowchart from RealSense acquisition through plane segmentation, K-means clustering, segment merging, watershed instance separation, regionprops pose extraction, and camera-to-base transformation.

IV. KINEMATICS AND MOTION PLANNING

A. Forward Kinematics

The arm is modelled using the DH convention. Each joint's coordinate frame is defined by four parameters (a_i , α_i , d_i , θ_i), and the individual joint transform is the standard 4×4 DH homogeneous matrix:

$$T_i = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

where c and s denote cosine and sine. For joint 1, with $a_1 = 0$, $\alpha_1 = \pi/2$, and $d_1 = 13.7$ cm, this rotates the frame about z by θ_1 and tilts the new z -axis into the vertical plane in which joints 2–4 operate. Joints 2–4 all have $\alpha = 0$ and $d = 0$, so each simplifies to a pure planar rotation and translation.

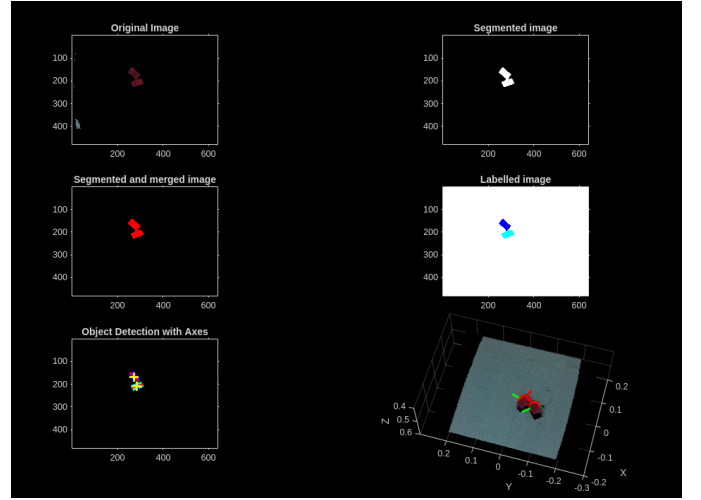


Fig. 5. Intermediate perception pipeline stages: (top-left) original image, (top-right) K-means segmented, (middle-left) segmented and merged, (middle-right) labeled instances, (bottom) object detection with 3D axes and point cloud visualization.

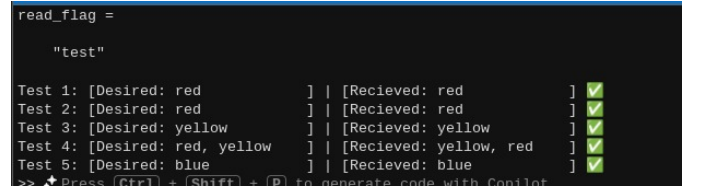


Fig. 6. Automated perception test output showing 5/5 correct color identifications across varied test scenes.

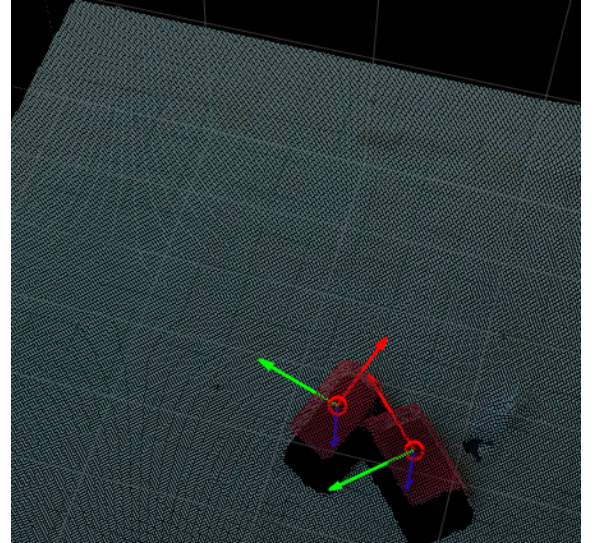


Fig. 7. Successful detection of touching blocks: 3D point cloud with overlaid coordinate axes at each detected centroid, demonstrating correct instance separation.

The cumulative end-effector pose is $\mathbf{T}_0^4 = \mathbf{T}_1 \cdot \mathbf{T}_2 \cdot \mathbf{T}_3 \cdot \mathbf{T}_4$. Because joints 2–4 share $\alpha = 0$, the arm reduces to a planar three-link mechanism. Defining planar reach r and height s :

$$r = a_2 c\theta_2 + a_3 c(\theta_2 + \theta_3) + a_4 c(\theta_2 + \theta_3 + \theta_4) \quad (4)$$

$$s = a_2 s\theta_2 + a_3 s(\theta_2 + \theta_3) + a_4 s(\theta_2 + \theta_3 + \theta_4) \quad (5)$$

the end-effector position in world coordinates is $x = r \cos \theta_1$, $y = r \sin \theta_1$, $z = s + d_1$. The full derivation is in Appendix A.

B. Inverse Kinematics

The analytical IK exploits the planar structure to decouple into five steps.

Step 1 — Base rotation:

$$\theta_1 \in \{\text{atan2}(y, x), \text{atan2}(y, x) + \pi\} \quad (6)$$

yielding two candidates (front- and back-facing).

Step 2 — Wrist decoupling: With horizontal reach $r = \sqrt{x^2 + y^2}$ and height $s = z - d_1$, the wrist centre is:

$$\bar{r} = r - a_4 \cos \varphi, \quad \bar{s} = s - a_4 \sin \varphi \quad (7)$$

Step 3 — Elbow angle via the law of cosines:

$$\cos \theta_3 = \frac{\bar{r}^2 + \bar{s}^2 - a_2^2 - a_3^2}{2 a_2 a_3} \quad (8)$$

If $|\cos \theta_3| > 1$, the target is unreachable. Otherwise $\theta_3 = \pm \arccos(\cos \theta_3)$ gives elbow-up and elbow-down branches (2 candidates per θ_1).

Step 4 — Shoulder angle: With $A = a_3 \sin \theta_3$ and $B = a_2 + a_3 \cos \theta_3$:

$$\theta_2 = \text{atan2}(\bar{s}, \bar{r}) - \text{atan2}(A, B) \quad (9)$$

(1 solution per (θ_1, θ_3) pair).

Step 5 — Wrist angle:

$$\theta_4 = \varphi - \theta_2 - \theta_3 \quad (10)$$

(1 solution per (θ_2, θ_3) pair).

Total: $2 \times 2 \times 1 \times 1 = 4$ candidate solutions. The full derivation is in Appendix B.

C. Solution Filtering and Selection

The function `findValidSolution.m` filters the raw IK candidates through three sequential safety checks:

- 1) **Joint limits** (`checkJointLimits.m`) — wraps each angle to $[-\pi, \pi]$ and rejects any solution where $\text{mod}(|\theta|, \pi) \geq \text{deg } 2\text{rad}(150)$, i.e., any joint within 30° of $\pm 180^\circ$.
- 2) **Self-collision** (`checkSelfCollision.m`) — linearly interpolates 20 waypoints along the joint-space path from the current configuration to the candidate and evaluates MATLAB's `checkCollision` on a `rigidBodyTree` model at each waypoint, returning immediately on the first collision detected.
- 3) **Floor collision** — evaluates forward kinematics and rejects solutions with end-effector $z < 0$.

From the surviving candidates, `findSolution.m` selects the one minimizing:

$$J = 0.3|\Delta\theta_1| + 0.3|\Delta\theta_2| + 0.2|\Delta\theta_3| + 0.2|\Delta\theta_4| \quad (11)$$

Joints 1 and 2 are penalized more heavily because they produce the largest physical displacements.

D. Motion and Gripping Strategy

Collision-free path. The motion strategy uses free-space traversal through a pre-grasp pose 5 cm vertically above the target. The arm translates laterally at the pre-grasp height, descends along the z -axis to the grasp pose, closes the gripper, and ascends back to the pre-grasp elevation before translating to the destination.

Approach angle. Two modes are supported. In orientation-controlled mode, $\varphi = \pi/2$ (straight-down). In position-only mode, φ is passed as NaN and auto-computed as:

$$\varphi = \text{atan2}(z - 24.2, \sqrt{x^2 + y^2}) \quad (12)$$

where 24.2 cm is the nominal shoulder height.

Grip timing. A 2-second pause is inserted before and after each grip command, allowing residual vibrations to decay.

Secure hold. A fixed joint angle for motor 5 forces the servo into its holding-torque regime, where the internal PID controller continuously drives against the object, preventing slip during transit.

E. State Machine

The function `moveByCoordinates.m` orchestrates five steps per Cartesian move: (1) receive target and resolve φ ; (2) compute IK candidates via `findJointAngles`; (3) filter through `findValidSolution`; (4) select via `findSolution`; (5) dispatch to the Arbotix controller or the digital twin via `animateInterpolation.m`.

A pick-and-place cycle composes two such sequences: `pickAndPlace.m` (pre-grasp $\rightarrow$ open gripper $\rightarrow$ descend $\rightarrow$ close gripper $\rightarrow$ retract) and `place.m` (pre-place $\rightarrow$ descend $\rightarrow$ open gripper $\rightarrow$ retract).

F. Pseudocode

Algorithm 1 summarises the pick-and-place procedure, and Algorithm 2 details the IK pipeline invoked at every `moveByCoordinates` call.

G. Key Functions

V. INTEGRATION AND RESULTS

A. Success Definition and Metrics

Success is defined as the physical transport of a cube from the pick location to the place location such that the final resting position of the cube lies within ± 10 mm of the commanded destination in both x and y , with no self-collisions or environmental collisions during the trajectory. This threshold was selected based on the physical gripper aperture: positional offsets beyond 10 mm cause the gripper jaws to miss the block entirely on the subsequent pick attempt. Two metrics are reported: **mean accuracy**, defined as the

Algorithm 1 Pick-and-Place Cycle

Require: Pick target (x_1, y_1, z_1) , place target (x_2, y_2, z_2) , pitch φ

- 1: **if** φ is NaN **then**
- 2: $\varphi \leftarrow \text{atan2}(z_1 - 24.2, \sqrt{x_1^2 + y_1^2})$
- 3: **end if**
- 4: — **Pick Phase** —
- 5: $\theta \leftarrow \text{SOLVEIK}(x_1, y_1, z_1 + 5, \varphi)$
- 6: Move to pre-grasp; open gripper; pause 2 s
- 7: $\theta \leftarrow \text{SOLVEIK}(x_1, y_1, z_1, \varphi)$
- 8: Descend to grasp; close gripper; pause 2 s
- 9: Retract to pre-grasp height
- 10: — **Place Phase** —
- 11: $\theta \leftarrow \text{SOLVEIK}(x_2, y_2, z_2 + 5, \varphi)$
- 12: Move to pre-place
- 13: $\theta \leftarrow \text{SOLVEIK}(x_2, y_2, z_2, \varphi)$
- 14: Descend to place; open gripper; pause 2 s
- 15: Retract to pre-place height

Algorithm 2 SOLVEIK — IK Pipeline (moveByCoordinates)

Require: Target (x, y, z, φ) , current configuration $\mathbf{q}_{\text{curr}}$

- 1: $\mathbf{Q} \leftarrow \text{findJointAngles}(x, y, z, \varphi)$ {up to 4 rows}
- 2: **if** $\mathbf{Q}$ is empty **then**
- 3: **abort:** target unreachable
- 4: **end if**
- 5: **for** each candidate $\mathbf{q}_i$ in $\mathbf{Q}$ **do**
- 6: **if** $\neg \text{checkJointLimits}(\mathbf{q}_i, \pm 150^\circ)$ **then**
- 7: discard $\mathbf{q}_i$; **continue**
- 8: **end if**
- 9: **if** $\text{checkSelfCollision}(\mathbf{q}_{\text{curr}}, \mathbf{q}_i, 20 \text{ pts})$ **then**
- 10: discard $\mathbf{q}_i$; **continue**
- 11: **end if**
- 12: **if** $\text{DH}(\mathbf{q}_i).z < 0$ **then**
- 13: discard $\mathbf{q}_i$; **continue**
- 14: **end if**
- 15: **end for**
- 16: $\mathbf{q}^* \leftarrow \arg \min_{\mathbf{q}_i} \sum_j w_j |\Delta \theta_j|$ { $\mathbf{w} = [0.3, 0.3, 0.2, 0.2]$ }
- 17: Send $\mathbf{q}^*$ to Arbotix (apply $+\pi/2$ offset on θ_2)
- 18: **return** $\mathbf{q}^*$

average Euclidean distance between the commanded place position and the measured landing position, and **repeatability**, defined as the standard deviation of the placement error across trials.

B. Testing Procedure

Testing was conducted by commanding repeated pick-and-place cycles across two coordinate sets spanning different workspace regions. Between trials, the block was manually reset to the pick location. The actual landing position was measured using the baseboard grid by visual inspection from directly above.

TABLE II
KEY PIPELINE FUNCTIONS

Function	Role
moveByCoordinates	Top-level Cartesian move orchestrator
findJointAngles	Analytical IK, up to 4 candidates
findValidSolution	3-stage safety filter
findSolution	Weighted-cost selection
DH	Forward kinematics (DH)
checkJointLimits	$\pm 150^\circ$ joint guard
checkSelfCollision	20-waypoint collision check
pickAndPlace	Pick-phase sequencer
place	Place-phase sequencer
animateInterpolation	Digital-twin animator

C. Quantitative Results**D. Quantitative Performance**

Coordinate Set 1 comprised seven trials along the primary workspace axes. Results are summarized in Table III.

TABLE III
PICK-AND-PLACE TEST LOG — SET 1

#	Pick (cm)	Target	Actual	Err (cm)
1	(20,0,2)	(0,21.75,2)	(0,20,2)	1.75
2	(20,0,2)	(0,19.75,2)	(0,20,2)	0.25
3	(0,19.75,2)	(20.25,0,2)	(20,0,2)	0.50
4	(20.25,0,2)	(0,19.75,2)	(0,20,2)	0.50
5	(19.75,0,2)	(20,0,2)	(20,0,2)	0.25
6	(20,0,2)	(0,19.5,2)	(0,20,2)	0.50
7	(19.5,0,2)	(0,19,2)	(20,0,2)	0.50

Mean accuracy: 6.07 mm. **Repeatability** (σ): 5.18 mm. **Maximum error:** 17.5 mm (trial 1). The first-trial outlier is attributed to a cold-start transient; the remaining six trials fall within the ± 10 mm threshold (2.5–5.0 mm). The low standard deviation indicates high repeatability: errors are largely systematic offsets rather than random scatter.

Coordinate Set 2, excluding one trial with a suspected data-entry error (sign inversion on the y -coordinate), yielded a mean accuracy of approximately 6 mm with $\sigma = 5.18$ mm, consistent with Set 1.

Table IV consolidates the accuracy and repeatability metrics across both coordinate sets.

TABLE IV
ACCURACY AND REPEATABILITY SUMMARY

	Mean (mm)	σ (mm)	Max (mm)
Set 1 ($n = 7$)	6.07	5.18	17.5
Set 2 ($n = 4^*$)	≈ 6.0	≈ 5.2	≈ 12.5
Combined	≈ 6.0	5.18	17.5

*One trial excluded due to suspected data-entry error.

Both sets fall within the ± 10 mm success criterion for the majority of trials, with the lone outlier (17.5 mm in Set 1, trial 1) attributable to a cold-start transient. The consistent standard deviation of ≈ 5 mm across both sets suggests that the dominant errors are systematic offsets rather than random

noise, and could be further reduced through a one-time calibration correction.

E. Theoretical vs. Actual Resolution

The AX-12A servos provide an angular resolution of 0.29° , which propagates through the FK at full extension (32 cm) to a theoretical spatial resolution of ≈ 1.4 mm. Observed errors (2.5–17.5 mm) are significantly above this floor, confirming that dominant error sources are mechanical rather than encoder-limited.

F. Sources of Error

Cumulative error propagation. Joint errors compound through the serial chain. At full extension, the base motor alone contributes $32 \text{ cm} \times \sin(0.29^\circ) \approx 1.6$ mm of positional uncertainty per encoder step.

Mechanical backlash. The AX-12A plastic gear trains introduce $\approx 1\text{--}2^\circ$ of slack, producing hysteresis-like error during direction changes.

Structural flex. Under load at full extension, the arm exhibits a slight mechanical droop not modelled by the rigid-body kinematics.

Human measurement error. Baseboard-grid measurement introduced parallax error in x and y , with no reference for z -axis height.

G. Stress Testing

Slightly incorrect pick coordinates were deliberately commanded, offsetting the pick position by several millimetres. The system generally picked the cube despite the offset, thanks to the gripper’s mechanical compliance. However, reliable grasping under offset conditions required a tighter motor-5 command, which triggered overload (red LED) when coordinates were accurate. No single motor-5 setting was optimal for both cases; an adaptive grip strategy using current feedback would be required for full robustness.

VI. DISCUSSION

A. Failure Analysis

Failure 1 — Motor vibrations. Significant oscillations during rapid joint motions risked object displacement. The default command speed excited the structural resonance of the aging hardware. The speed was reduced to 100 (Arbotix units), and 2-second grip pauses were introduced. Vibrations persist; increasing the derivative (D) gain on the AX-12A PID via the Dynamixel protocol could further damp overshoot without requiring a speed reduction.

Failure 2 — IK failures at workspace boundaries. Conservative joint limits rejected valid solutions near the workspace edges. Widening to the full $\pm 150^\circ$ range recovered most IK solutions.

Failure 3 — Gripper calibration. A fixed motor-5 command could not accommodate slight cube-size variation. The command was tuned to a compromise; adaptive grip via current sensing would resolve this.

Failure 4 — Sim-to-real divergence. The digital twin’s zero configuration did not match the hardware due to the $+\pi/2$ firmware offset on joint 2. Mirroring this offset in the simulation code resolved the discrepancy.

B. Design Evolution

The system evolved substantially through iterative testing. The perception pipeline began as a pure RGB approach using geometry- and area-based filtering, which proved fragile—background noise caused missed detections. Shifting to K-means over-segmentation followed by reference-color merging dramatically improved robustness.

The IK solver was initially configured with conservative joint limits well below $\pm 150^\circ$. This rejected too many valid solutions near workspace boundaries. The realization that the full rated range was both necessary and acceptable was a pivotal moment.

The digital twin, initially a visualization convenience, became essential for debugging the firmware offset issue. Comparing simulated and physical configurations side by side revealed a discrepancy that would have been extremely difficult to diagnose otherwise.

The departure of the third team member required redistributing all integration and testing responsibilities between two people.

C. Theory vs. Practice

The theoretical workspace (cylindrical, $r \approx 32$ cm) contracts to $r \approx 21$ cm under the fixed $\varphi = \pi/2$ constraint. A variable approach angle would recover much of this lost reach. The theoretical spatial resolution (1.4 mm) is $4\text{--}7\times$ smaller than observed errors (6–10 mm), with the gap attributable to backlash, flex, and zero-offset biases absent from the rigid-body model. The computed payload capacity (246.6 g) closely matches the specification (250 g), but aging hardware exhibited torque drops at loads well below this limit, particularly under low battery voltage.

VII. CONCLUSION AND FUTURE WORK

This project demonstrated a complete pick-and-place pipeline integrating perception (K-means + watershed + point-cloud localization), kinematics (DH-based FK and closed-form IK), collision-aware motion planning (three-stage filtering with weighted cost selection), and a real-time digital twin.

The system achieved a mean placement accuracy of 6.07 mm within the ± 10 mm threshold, with high repeatability ($\sigma = 5.18$ mm) across seven trials. The perception subsystem correctly identified all target colors across five validation scenes. The most impactful design decisions were the over-segment-then-merge perception strategy, the $\pm 150^\circ$ joint-limit expansion, and the 2-second grip pauses for vibration damping.

For future work: progressive φ -relaxation (iterating downward from $\pi/2$ until a valid IK solution is found) would extend workspace coverage. A closed-loop hardware-synchronization

mechanism [4] polling joint positions would eliminate open-loop timing race conditions. PID D-gain tuning could damp motor vibrations without slower speeds. On the perception side, real-time re-perception between picks, adaptive HSV thresholding, and edge-detection methods for stacked-cube separation are promising directions.

DIGITAL MATERIAL

- GitHub Repository: <https://github.com/YOUR-REPO>
- Demonstration Videos: <https://youtube.com/shorts/SEbUP5o99QI>

ACKNOWLEDGMENTS

The authors thank Dr. Basit Memon for his guidance throughout the semester and the lab handbook that served as the blueprint for this system [4]. Shaheer bin Tariq was originally a member of the team but departed during the project; all final integration, testing, and reporting were completed by the remaining two authors. Generative AI tools were used for L^AT_EX formatting and technical rephrasing; all engineering logic, kinematic derivations, perception algorithms, and experimental results are the original work of the authors.

REFERENCES

- [1] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.
- [2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Pearson Prentice Hall, 2005.
- [3] Robotis, “AX-12A Dynamixel Servo Technical Specification,” [Online]. Available: <https://emanual.robotis.com/docs/en/dxl/ax/ax-12a/>. [Accessed: May 2026].
- [4] B. Memon, *EE366L/CE366L: Introduction to Robotics Lab Handbook*, 2nd ed. (v2.2). Habib University, Spring 2026.
- [5] MathWorks, “MATLAB Robotics System Toolbox Documentation,” [Online]. Available: <https://www.mathworks.com/products/robotics.html>. [Accessed: May 2026].
- [6] H. T. Caceres, “PhantomX Pincher Specifications,” ResearchGate, 2018. [Online]. Available: <https://www.researchgate.net/publication/32222351>. [Accessed: Apr. 2026].
- [7] Intel RealSense, “Intel RealSense SDK 2.0 Documentation,” [Online]. Available: <https://www.intelrealsense.com/sdk-2/>. [Accessed: May 2026].

APPENDIX A

FORWARD KINEMATICS DERIVATION

The PhantomX Pincher is modelled using the DH convention with the parameters in Table I. Each joint transform follows (3).

Joint 1 transform (T_1): With $a_1 = 0$, $\alpha_1 = \pi/2$, $d_1 = 13.7$:

$$T_1 = \begin{bmatrix} c\theta_1 & 0 & s\theta_1 & 0 \\ s\theta_1 & 0 & -c\theta_1 & 0 \\ 0 & 1 & 0 & 13.7 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (13)$$

This rotates the frame about z by θ_1 and tilts the new z -axis into the vertical plane.

Joints 2–4: Since $\alpha = 0$ and $d = 0$, each simplifies to:

$$T_i = \begin{bmatrix} c\theta_i & -s\theta_i & 0 & a_i c\theta_i \\ s\theta_i & c\theta_i & 0 & a_i s\theta_i \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad i = 2, 3, 4 \quad (14)$$

Cumulative transform: $T_0^4 = T_1 \cdot T_2 \cdot T_3 \cdot T_4$. Because joints 2–4 share $\alpha = 0$, the arm reduces to a planar three-link mechanism in the vertical plane set by θ_1 :

$$r = a_2 c\theta_2 + a_3 c(\theta_2 + \theta_3) + a_4 c(\theta_2 + \theta_3 + \theta_4) \quad (15)$$

$$s = a_2 s\theta_2 + a_3 s(\theta_2 + \theta_3) + a_4 s(\theta_2 + \theta_3 + \theta_4) \quad (16)$$

End-effector position: $x = r \cos \theta_1$, $y = r \sin \theta_1$, $z = s + d_1$.

APPENDIX B

INVERSE KINEMATICS DERIVATION

The IK computes $(\theta_1, \theta_2, \theta_3, \theta_4)$ for a desired (x, y, z) with pitch $\varphi = \theta_2 + \theta_3 + \theta_4$.

Solving for θ_1 : $\theta_1 = \text{atan2}(y, x)$, with a back-facing candidate $\theta_1' = \theta_1 + \pi$ (2 candidates).

Reduced variables: $r = \sqrt{x^2 + y^2}$, $s = z - d_1$. Wrist centre: $\bar{r} = r - a_4 \cos \varphi$, $\bar{s} = s - a_4 \sin \varphi$.

Solving for θ_3 : $\cos \theta_3 = (\bar{r}^2 + \bar{s}^2 - a_2^2 - a_3^2) / (2a_2 a_3)$. If $|\cos \theta_3| > 1$, unreachable. Otherwise $\theta_3 = \pm \arccos(\cos \theta_3)$ (2 candidates per θ_1).

Solving for θ_2 : Define $A = a_3 \sin \theta_3$, $B = a_2 + a_3 \cos \theta_3$. Then:

$$\sin \theta_2 = \frac{B\bar{r} - A\bar{s}}{A^2 + B^2} \quad (17)$$

$$\cos \theta_2 = \frac{A\bar{r} + B\bar{s}}{A^2 + B^2} \quad (18)$$

$\theta_2 = \text{atan2}(\sin \theta_2, \cos \theta_2)$, equivalently $\theta_2 = \text{atan2}(\bar{s}, \bar{r}) - \text{atan2}(A, B)$. (1 solution per (θ_1, θ_3) .)

Solving for θ_4 : $\theta_4 = \varphi - \theta_2 - \theta_3$ (1 solution per (θ_2, θ_3)).

Total: $2 \times 2 \times 1 \times 1 = 4$ candidates, filtered by `findValidSolution` and ranked by (11).

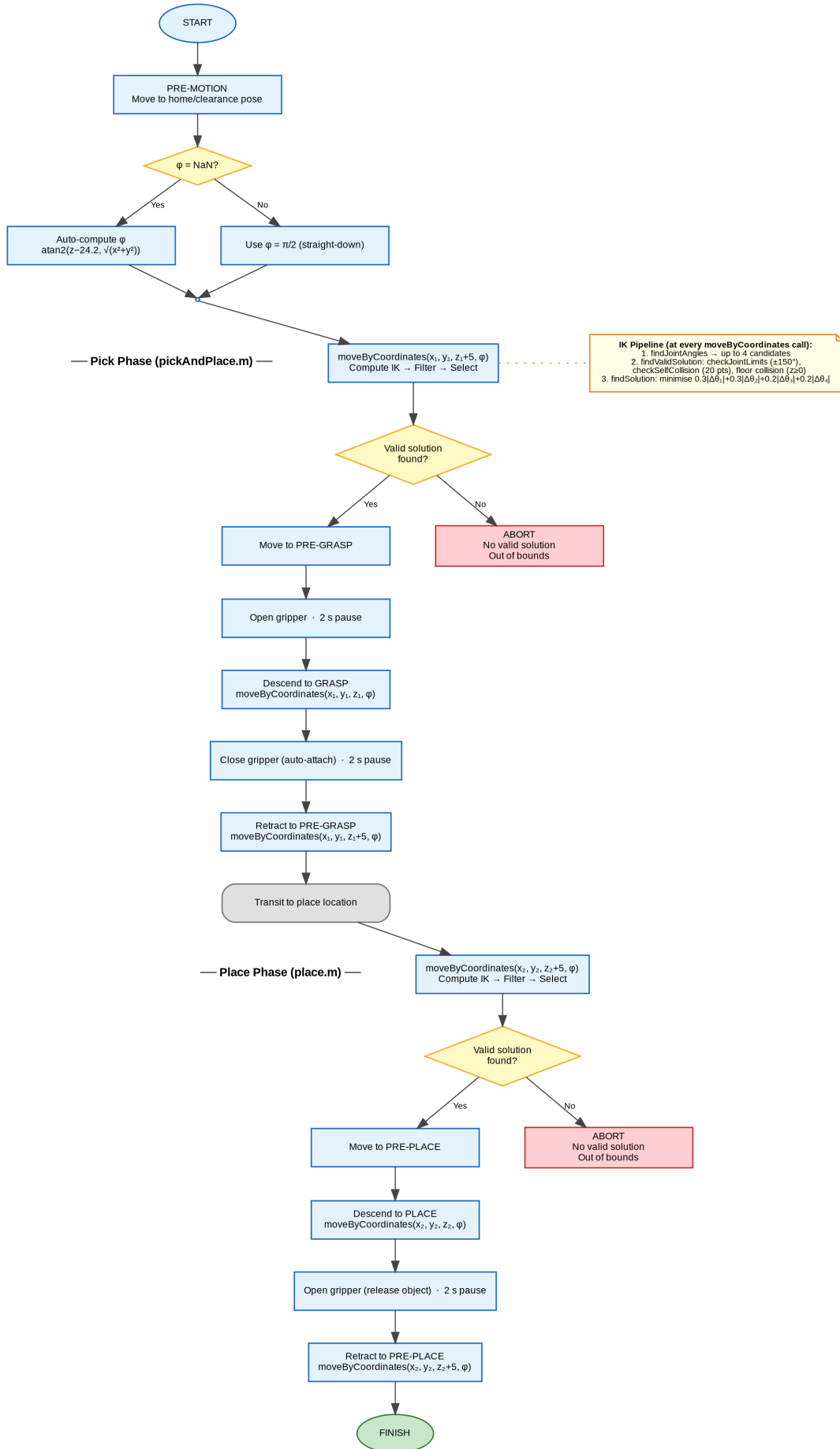


Fig. 8. Pick-and-place motion pipeline. The pick phase (pickAndPlace.m) and place phase (place.m) each execute the full IK → filter → select → dispatch sequence at every moveByCoordinates call. The annotation box details the three-stage filtering and cost function applied at each call. Red nodes indicate abort on unreachable targets.